Parallele Programmierung

Bsc. Informatik ETH Zürich

Laurin Seeholzer

April 2, 2026

Contents

1 Threads and Processes **3**
- 1.1 Processes . . . 3
- 1.2 Parallelism . . . 3
- 1.3 Multithreading . . . 4

2 Synchronization **7**
- 2.1 Locks and Synchronized . . . 7
- 2.2 Wait, notify, notifyAll . . . 8
- 2.3 Lock API . . . 8

3 Optimization **11**
- 3.1 Caching . . . 11
- 3.2 Vectorization . . . 11
- 3.3 Instruction Level Parallelism (ILP) . . . 11
- 3.4 Pipelining . . . 12
- 3.5 Parallel Performance . . . 12

4 Divide and Conquer **14**
- 4.1 Executor Service . . . 14
- 4.2 Fork-Join . . . 15
- 4.3 Task graphs . . . 16

5 Parallel Algorithms **17**
- 5.1 Parallel Patterns . . . 17
- 5.2 Parallel Algorithms . . . 17
  - 5.2.1 Prefix-Sum . . . 17
  - 5.2.2 Pack . . . 18
  - 5.2.3 Quicksort . . . 18

6 Shared Resources **19**
- 6.1 Race Conditions . . . 19
- 6.2 Guidelines for Concurrent programming . . . 19

1 Threads and Processes

1.1 Processes

Definition 1.1. *A process is a program executing inside an OS (operating-system). Processes share CPU but have seperate memory space.*

Definition 1.2. *The **context** of a process is the set of utilities it uses and or operates on. That is:*

- ***Hardware context:*** *Instruction counter, Registers, Execution State, Flags ...*
- ***Memory context:*** *Virtual Memory, Heap-Space, Stack ...*
- ***OS-level context:*** *Process ID, open files ...*

The lifecylce of a process is managed by the OS and can be described by the following states:

- **CREATED:** The process is loaded from the disc
- **WAITING:** The process is waiting in the main-memory for execution.
- **RUNNING:** The process is executed
- **BLOCKED:** The process is waiting for access to I/O devices
- **TERMINATED:** The process has finished its execution

Definition 1.3. ***Context Switch:*** *The process of saving the context of a process and loading the context of another process. This happens for example when a process recives the RUNNING state and another process gets BLOCKED.*

All processes are managed by the OS, which schedules, synchronizes, starts and terminates processes, allows comminication and manages ressources. The OS uses so called Process control blocks (PCB).

Definition 1.4. *A **process-control-block (PSB)** is a table-entry storing relevant process information such as Program counter, Registers, Adress Space, Open files ...*

1.2 Parallelism

Definition 1.5. ***Parallelism*** *is when multiple processes execute simultaneously on diffrent CPU-Cores.*

Definition 1.6. ***Concurrency*** *is when multiple processes execute interleaved on a single CPU-Core.*

Remark 1.7. *Observe that parallelism implies concurrency, but concurrency does not imply parallelism.*

1.3 Multithreading

Definition 1.8. *A thread is an independent sequence of exection running in the same OS-process.*

Remark 1.9. *While* ***processes can only communicate through main-memory*** *and have diffrent memory spaces,* ***threads share the same adress space****. This leads to faster and more efficient context-switches as adressspace and PCB loading is not necessary.*

There are diffrent levels of threads:

- **User-Level Threads:** Threads managed by the application using a thread library, ex. Java-Threads managed by JVM
- **Kernel-Level Threads:** Threads managed by to OS, which can schedule them on diffrent cores.
- **CPU-Level Threads:** Threads managed by the CPU, which can schedule them on the physical cores. Ex. A CPU might have 4 physical cores, which apear to the OS as 8 logical cores.

There are two ways to create a thread in java. We can directly extend the Thread class and implement our run method in the Thread.

Code Example 1.10

```
class MyThread extends Thread {
    ...
    public void run() {
        ...
    }
}
MyThread thread = new MyThread();
thread.start(); // calls MyThread.run()
```

Or we can use the Runnable Interface to create a runnable object which we pass to the Thread.

Code Example 1.11

```java
class MyRunnable implements Runnable {
    ...
    public void run() {
        ...
    }
}
MyRunnable runnable = new MyRunnable();
Thread thread = new Thread(runnable);
thread.start(); // calls MyRunnable.run()
```

In its lifecylce a thread can have the follwoing states:

- **NEW:** The thread has been created but not yet started.
- **RUNNABLE:** The thread is ready to run and waiting for the OS to schedule it.
- **RUNNING:** The thread is currently executing.
- **BLOCKED:** The thread is waiting for access to I/O devices
- **WAITING:** The thread is waiting to be notified
- **TERMINATED:** The thread has finished its execution

> **Remark 1.12.** *Every Thread has an **ID, Name, Priority and State** which might be usfull for certain tasks.*

To wait for a thread to finish, we can use the .join() method on the thread we want to wait on. This lets the main-thread sleep until the joined thread terminates and wakes it up again.

Code Example 1.13

```java
Thread t = new MyThread();
t.start(); //start t
t.join(); //sleep until t is finished
continueWork(); // this is only called after t finishes
```

> **Remark 1.14.** *Using the **.join() method might not always be the best option**, as it produceses a **significant overhead**. If the execution time of a thread is short, it might be better to let the main-thread busy wait in a while loop.*

> **Important 1.15.** *If a worker-thread throws an exception the main-thread will be woken up, but it **will not know of the error**, which might lead to false results. To make sure we don't produce false results we have to **set an UncaughtExeption-Handler** using a try-catch block when joining threads. The same holds for **threads that run indefinetly** we need to make sure to **have a timer task** that interupts threads or manually interupt them.*

Code Example 1.16

```
Thread t = new MyThread(); //worker thread
Thread timer = new MyTimerThread(); //timer thread

t.start();
timer.start(t, 1000);

try {
    t.join();
} catch (UncaughtExeptionHandler) {
    System.out.println("Error in thread t"); //handle the error
}
continueWork();

MyTimerThread extends Thread {
    run(Thread thread, int time) {
        timer.sleep(time);
        thread.interupt(); //interrupt indefinetly running thread
    }
}
```

Important 1.17. ***Thread scheduling is not deterministic.*** *This means that we can't rely on the order (interleaving) in which threads are executed.*

2 Synchronization

Example 2.1. *A famous example of thread-safety issues is the counter. When a counter c is incremented using c++ or ++c the value of c first gets read, then incremented and then stored. The counter fails if one thread stores his result between the time another reads and stores again.*

Definition 2.2. *A **Bad Interleaving** is an interleaving of operations from multiple threads that leads to an incorrect program state.*

Definition 2.3. *A **Critical Section** is a codeblock that might lead to bad interleavings. To ensure program correctness we need the enforce mutual exclusion on them.*

Definition 2.4. ***Thread safty** refers to nothing bad happeing in all possible interleavings and thus implies program safety.*

Definition 2.5. ***Thread Liveness** refers to something good happening eventually, which implies the correctness of a program.*

2.1 Locks and Synchronized

Every Java-Object has an **intrinsic lock or lock monitor**. These locks are unique and can be optaind by threads which allows us to lock a certain codeblock. If a thread wants to access such a codeblock and execute the code within it first has to optain the lock, while it has the lock, no other thread can accuire it.

Code Example 2.6

```
synchronized(lockObject) {
    i++;
}
```

This ensures mutual exlusion on the synchronized codeblock. It is also possible to use synchronized as a keyword in a function to sync. the whole function.

Remark 2.7. *By default synchronized blocks use "this" as the lock-object and unlock when an exception occures.*

Remark 2.8. *Java-Locks are **reentrant**, meaning they can be locked multiple times*

> *by the same thread recursively (it will then have to release the lock as many times as it acquired it).*

For incrementing and decrementing integers there is a special DataType AtomicInteger which has built in synchronization as incrementing/decrementing is done in a single step.

Code Example 2.9

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();
```

2.2 Wait, notify, notifyAll

We can use wait() on a lock-object to tell the current thread to sleep until further notice if the lock is not available. This allows use to save ressources.

Code Example 2.10

```
synchronized(lockObject) {
    while(!condition) {
        lockObject.wait();
    }
    // do work
}
```

To wake one or multiple threads and tell them that the lock is now available we can use notify() or notifyAll() on the lock-object to wake up an aribtrary waiting thread or all of them.

Important 2.11. *Always **wait inside a while loop**, as this ensures checking the condition after beeing notified.*

Code Example 2.12

```
synchronized(lockObject) {
    while (!condition) {
        lockObject.wait();
    }
    //do work
    notifyAll();
}
```

2.3 Lock API

Example 2.13. *The problem with the below code is that it is **not thread-safe**. If two threads try to withdraw money at the same time, they might both read the same balance and then both write to it, resulting in a wrong or even negative final balance. We need mutual exclusion in that critical section.*

Code Example 2.14

```
class BankAccount {
    private int balance;

    int balance() {return balance;}

    void setBalance (in x) {balance = x;}

    void withdraw(int amount) {
        int b = balance();
        if (b less than amount) {
            throw new Exception("Not enough money");
        }
        setBalance(b-amount);
    }
}
```

Instead of using built-in synchronized blocks we can also use locks from the java.util.concurrenct.locks package.

Code Example 2.15

```
class BankAccount {
    private Lock lock = new ReentrantLock();
    ...
    void withdraw(int amount) {
        lock.lock();
        try {
            ...
        } finally {
            lock.unlock();
        }
    }
}
```

Important 2.16. *Always use a try-finally block to ensure the lock is released even if an error occures.*

Remark 2.17. *There is also a* ***lock.tryLock(100, TimeUnit.MILLISECONDS)*** *method which tries to accuire the lock only for a given amount of time. This is useful for avoiding deadlocks (ex. when two threads have each one lock and wait on the other to get released)*

Aspect	**synchronized**	**Lock API**
Lock handling	Automatic (JVM managed)	Manual (lock() / unlock())
Lock release	Automatic	Must be done explicitly (finally!)
Scope	Method or block scoped	Flexible (can span multiple methods)
Waiting behavior	Thread blocks indefinitely	tryLock() avoids blocking
Interruptibility	Not interruptible while waiting	lockInterruptibly() supports interrupt
Flexibility	Low	High
Complexity	Simple, less error-prone	More complex, error-prone
Deadlock risk	Lower	Higher (if unlock forgotten)
Typical use	Simple synchronization	Advanced concurrency control

3 Optimization

Theorem 3.1. ***Moore's Law*** *states that the number of transistors on a microchip doubles approximately every two years.*

By moores law architects improved sequential execution every year. Sequential programs were becoming exponentialy faster. But in the early 2000's we began hitting walls:

- **Power-Consumption**: Expensive cooling limited further increase of clock-speed.
- **Memory-Wall**: CPU's became faster than memory and had to wait (stall) on load/store operations.

3.1 Caching

Faster CPU's does not mean faster execution. The CPU might compute almost instantly but the loading and storing of the exectution data/result forms a bottleneck, as this is one of the slowest operations (high latency).

Definition 3.2. ***Cache*** *is a fast memory that preloads and stores frequently used data form the memory to reduce latency.*

The usage of cache reduces stalls (times in which the CPU can not execute as the data is not yet present).

> **Remark 3.3.** *Programmers can influence the cache usage by organizing their data in a cache-friendly way. That means increasing the locality of data access so that the cache can efficiently prefetch data.*

3.2 Vectorization

With vectorized data we can use SIMD to exectute the same operation on multiple data points all at once.

Definition 3.4. ***SIMD*** *(Single Instruction Multiple Data) is a type of parallel processing where the same instruction is applied to multiple data points simultaneously by using multiple ALUs inside the CPU. The same instruction gets broadcasted to multiple ALU which execute in parallel.*

3.3 Instruction Level Parallelism (ILP)

Definition 3.5. ***Superscalar execution*** *refers to out of order execution of independent instructions in parallel using multiple ALUs.*

Definition 3.6. ***Speculative execution*** *refers to the execution of instructions before the actual need for them is known.*

3.4 Pipelining

Pipelining increases throughput by overlapping the execution of multiple instructions in diffrent stages.

Example 3.7. *As soon as I am finished washing my laundry and start hanging it, my neighbour can already use the machine and start wahsing his clothes. My neighbour does not have to wait for me to wash, hang, dry, fold, store my clothes.*

Definition 3.8. ***Throughput*** *is the amout of work that can be done by a system in a given period of time. It can be bounded by*

$$tp = \frac{1}{max(computationTime(stages))}$$

Definition 3.9. ***Latency*** *is the time needed to perform a given computation. It can be bound by*

$$L = numStages * max(computationTime(stages))$$

Definition 3.10. *A pipeline is* ***balanced*** *if all stages take the same amount of time.*

Balancing a pipeline can increase throughput.

Example 3.11. *Assume washing is the stage that takes the longest. Having two small washing machines (one to prewash, one to rinse) instead of a large one can increase through put as now my neighbour can start prewashing as soon as I start rinsing and my neighbours neighbour can start prewahsing as soon as i am finished rinsing. This balances the pipeline.*

3.5 Parallel Performance

Definition 3.12. ***Scalability*** *describes how performance improves with additional resources (cores, memory, etc.).*

Definition 3.13. ***Speedup*** *is the ratio of the execution time of the sequential algorithm to the execution time of the parallel algorithm.*

$$S_p = \frac{T_1}{T_p}$$

Definition 3.14. ***Efficiency*** *is the ratio of speedup to the number of processors.*

$$E_p = \frac{S_p}{p}$$

Remark 3.15. *Speedup depends on the definition of T_1 (**Baseline**), is it the execution time of the optimized sequential algorithm or the unoptimized one?*

Theorem 3.16. *Amdahl's Law sets a limit on speedup, as the sequential part of the algorithm can not be parallelized.*

$$S(p) = \frac{1}{1 - f + \frac{f}{p}}$$

Theorem 3.17. *Gustafson's Law proposes an optimistic view compared to Amdahl. It shows that we might not be able to shrink execution time but we can perform more operations in that same execution time.*

$$S_p = f + P(1 - f)$$

4 Divide and Conquer

Definition 4.1. ***Divide and Conquer*** *refers to the strategy of splitting a problem into smaller subproblems, solving them recursively, and combining the results.*

This concept can be used in many parallel algorithms. Until now we have used manual forking .start() and joining .join() of Threads. We can continue using manual fork/join with DC, but this can lead to several Problems.

- Poor reusability of code
- We need to commit to a number of threads a priori and can not dynamically adjust.
- Subproblems are not automatically assigned to a thread.

Remark 4.2. *There are some fixes that can improve manual fork/join such as* ***sequential-cutoffs*** *or running one subproblem on the thread itself (****invoke/compute****), instad of creating two new threads.*

4.1 Executor Service

The Executor Service framework provides the way to schedule and run tasks on a thread-pool automatically.

Definition 4.3. ***Runnable****: A task without a return value. Passed to execute() (fire-and-forget) or submit() (returns an empty future).*

Definition 4.4. ***Callable****: A task that returns a result of given type T and can throw checked exceptions. Passed to submit(Callable) and returns a Future of type T.*

Remark 4.5. *There are diffrent* ***Pool Types****: Executors.newFixedThreadPool(n) keeps n worker threads alive. newCachedThreadPool() scales up and down dynamically depending on load.*

Code Example 4.6

```
ExecutorService executor = Executors.newFixedThreadPool(4);

class MyRunnable implements Runnable {
    public void run() {...}
}

class MyCallable implements Callable<Integer> {
    public Integer call() throws Exception {...}
}
```

```

executor.execute(new MyRunnable());

Future<Integer> future = executor.submit(new MyCallable());
int res = future.get(); // Blocks until result is returned

executor.shutdown();
```

Pros and Cons of Executor service:

- Not suited for: Recursive subproblems that depend on each other and have a deep fork-tree. As the service will eventually run out of threads.
- Suited for: flat structures that can run independently in parallel.

4.2 Fork-Join

The fork-join framework is built for divide and conquer parallelism. It manages a pool of threads using a **work-stealing strategy**: each thread maintains a local task-queue, and when finished with its own tasks, steals work from others to minimize idle time.

Definition 4.7. ***RecursiveTask****: A task that returns a result of type given Type T (equivalent to Callable).*

Definition 4.8. ***RecursiveAction****: A task that does not return a result (equivalent to Runnable).*

Key Methods of the Fork-Join Framework:

- **fork()**: Asynchronously pushes the task to the local thread's queue.
- **join()**: Blocks until the result is ready. While waiting, the thread does not truly idle but uses work-stealing to execute other tasks.
- **compute()**: The main logic method that must be overridden.

Code Example 4.9

```
class MyTask extends RecursiveTask<Integer> {
    int start, end;
    MyTask(int start, int end) { this.start = start; this.end = end
        ; }

    protected Integer compute() {
        if (end - start <= SEQUENTIAL_CUTOFF) {
            return baseCase();
        } else {
            int mid = (start + end) / 2;
            MyTask left = new MyTask(start, mid);
            MyTask right = new MyTask(mid, end);

```

```
            left.fork(); // Async execute left half
            int rightRes = right.compute(); // Optimize: caller thread computes right half
            int leftRes = left.join();      // Wait for left half

            return leftRes + rightRes;
        }
    }
}
```

Important 4.10. *Make sure that every task has enough work to do (aprox. 100-10'000 ops.), else the overhead will be larger than the parallel speedup gain.*

4.3 Task graphs

The execution of parallel programs using Fork-Join can be represented as a Task-Graph (DAG).

Definition 4.11. *We introduce a new size T_∞ which is the execution time assuming unlimited hardware. This is called the* ***span*** *and can be visualized as the sum of the execution times of the longest dependent path in our DAG.*

Remark 4.12. *The maximum speedup is limited by $\frac{T_1}{T_\infty}(AmdahlsLaw)$.*

It is the programmers responsibility to find the optimal decomposition/alogrithm for the problem to ensure a minimal span (a balanced tree). **Fork-join will guarantee optimal scheduling of the tasks with minimal idletime.**

5 Parallel Algorithms

5.1 Parallel Patterns

Definition 5.1. *A **reduction** is a parallel operation that produces a single answer from a collection via an associative operator using for example DC. (The order of operations does not matter).*

Example 5.2. *Finding the Max, Min, Sum or Product over all entries of an array.*

Definition 5.3. *A **map** applies a function to each elements of a collection producing an output of the same size.*

Example 5.4. *Vector addition can be done using a map pattern. We split the vectors into dim(v)/2 recursively until we have dim(v)=1 then sum the vectors and write the result directly into the result vector.*

> **Remark 5.5.** *Maps and Reductions have (when parallelized) an optimal span of* $O(\log n)$.

Definition 5.6. *A **stencil** is an extension of the map pattern that applies a function to each element of a collection producing an output of the same size. The function uses the element itself and its neighbours.*

Example 5.7. *A smoothing kernel (ex. 3x3) that calculates the average value is sled over an image to smooth/blur the image.*

5.2 Parallel Algorithms

5.2.1 Prefix-Sum

> **Remark 5.8.** *Prefix-Sum (calculating input[1] + input[2] + ... + input[n]) seams to be unparallisable. But infact it is!*

Algorithm 5.9

```
upTask(); //divide problem and build tree with sums of subproblems
downTask(); //fill in the correction values (left_sum +
    parent_correction)
```

Theorem 5.10. *The span of the prefix sum algorithm is* $O(\log n)$. *This leads to a*

parallelism of $\frac{n}{\log n}$.

5.2.2 Pack

Another application of this problem is the **Pack** algorithm which removes all elements from an array that satisfy a certain condition and packs them into a seperate output.

Algorithm 5.11

```
applyMap(); //applies a map to the input
//results in a binary vector [1, 0, 1, 1, 0, ...]

parallelPrefixSum(); //prefix sum of the binary vector
//results in [1, 1, 2, 3, 3, ...]

applyPackMap(); //applies map to the result of the prefix sum
//results in [0, 2, 3, ...] (indices of output elements)
```

5.2.3 Quicksort

We can reduce the limiting factor of the problem division from a span of $O(n)$ to a span of $O(\log n)$ using the pack-pattern we have seen before.

Algorithm 5.12

```
applyMaps(); //apply maps for greater and smaller than pivot
parallelPrefixSum(); //num. of smaler/greater elems. come bofore
applyMap(); //index = numLessThan || numTotalLessThan +
    numGreaterThan
```

We can do this by using two pack-patterns to partition the data. This leads to a partitioning of $O(\log n)$ levels times $O(\log n)$ for prefixSum hence $O(\log^2 n)$ total span. This means a parallelism of $\frac{n \log n}{\log^2 n} = \frac{n}{\log n}$.

6 Shared Resources

6.1 Race Conditions

Definition 6.1. *A **Race condition** occurs in concurent programming when the correctness of the system depends on the specific interleaving or ordering of execution of operations.*

Definition 6.2. *A **Low-level race condition** is a race condition that occurs because of insufficient synchronization of memory access (Data Races).*

Definition 6.3. *A **High-level race condition** is a race condition that occurs because of a wrong decomposition/algorithm. The program might ensure mutual exclusion on the shared resources but the program might still produce wrong results due to **bad interleavings**.*

Generally we want one of three things when it comes to shared ressources:

- **Thread local (Isolated Mutability):** We dont use the ressource of other threads.
- **Read only (Immutable):** We only read and do not change the ressource
- **Mutual exclusion (Synchronization):** We use the ressource of other threads but we ensure that only one thread accesses it at a time.

6.2 Guidelines for Concurrent programming

Important 6.4. ***Thread-Locality:*** *Minimize the sharing of mutable state between threads. Only share ressources if needed.*

Important 6.5. ***Immutability:*** *Use immutable data structures whenever possible. They are inherently thread-safe.*

Important 6.6. ***Consitent locking:*** *If you need to share mutable state, use locks to ensure mutual exclusion.*

Definition 6.7. ***Lock granularity*** *refers to the amount of code that is proteted by a lock in ways:*

- *Size of synchronized blocks*

- *Number of objects a lock is used for*

Theorem 6.8. ***Coarse-grained locking*** *is easier to implement but leads to less concurrency and therfore performance loss.* ***Fine-grained locking*** *is harder to implement without creating bugs but leads to more concurrency and therfore performance gain.*

Important 6.9. ***Start with coarse-grained (simpler)*** *and move to fine-grained (performance) only if contention on the coarser locks becomes an issue.*

Important 6.10. *Do not do expensive computations or I/O in critical sections, but also don't introduce race conditions*

Definition 6.11. ***Atomicity:*** *An operation is atomic if it is performed as a single, indivisible unit. In other words, it either completes entirely or not at all, without any intermediate states being visible to other threads.*